

Automation of Order Processing

A report on building a multimodal order-processing prototype for Foppa GmbH

Natalia Akatyeva

NLP & Web Analytics · M2 2025/26

Free University of Bozen-Bolzano · June 2026

Project context

This project was focused on building an AI-assisted prototype for Foppa GmbH, a food wholesaler in South Tyrol that supplies restaurants and hotels. The company already has structured internal systems and an ERP environment, but a large part of customer communication still happens through informal channels. Many customers do not send orders through a standardized digital form. Instead, they use WhatsApp messages, photos of handwritten notes, scanned PDFs, old fax-style order sheets, and sometimes short free-text messages.

This creates a practical problem for the back office. A customer order may be easy for a human to understand in context, but it is not directly usable by an ERP system. The ERP needs exact product codes, standardized product names, clean quantities, and correct units. A message such as “2 Kübl Naturjoghurt Brimi” or “Burro mini porzioni” has to be translated into a specific item from the catalogue. This requires not only reading the message, but also understanding informal wording, multilingual terms, abbreviations, and product names that may differ from the official catalogue description.

The goal of my project was to build a prototype that reduces this manual work. The system should accept a raw customer order, extract the relevant order items, match them with the correct products in Foppa’s catalogue, and return a structured JSON output that can be reviewed by a human before being transferred to the ERP. The project was not intended to fully replace human validation. Instead, the idea was to create a back-office assistant that prepares a reliable draft and highlights uncertain cases.

Extraction with Gemini

The input data included text messages, images, PDFs, and scans. Orders were often written in a mix of German, Italian, and local dialects, and could include abbreviations, spelling variations, compound words, and delivery notes mixed with

actual product lines. To convert these diverse inputs into structured JSON, I used Google's multimodal model Gemini 2.5 Flash. Gemini processes the input file or text and returns structured JSON containing raw item phrases, quantities, and additional fields such as delivery notes. This ensures that the system does not simply extract all visible text, but also separates actual order items from comments or logistics instructions.

To improve extraction accuracy, I implemented input-type-specific prompt instructions. PDFs were guided for table parsing and multi-page extraction, images received instructions for handling unclear handwriting, crossed-out items, and maintaining original spelling when uncertain, and plain-text messages were processed with line-by-line parsing rules. This approach stabilized the extraction layer by providing the model with context tailored to each type of input.

Matching

The first stage uses the customer template. Instead of immediately searching the full catalogue, the system first looks at the products that this customer has ordered before. This reduces the search space from 6,460 products to roughly 50 to 100 products for a typical customer.

The second stage applies fuzzy scoring. I used rapidfuzz because the original string comparison approach was not strong enough for real catalogue data. Basic string matching often fails when words are reordered or when product names are written in different forms. For example, "Naturjoghurt Brimi" and "BRIMI NATUR JOGHURT" should clearly be close matches, even if the word order is different. Also, matching engine combines several rapidfuzz metrics: `token_set_ratio`, which works well when word order is not important, `token_sort_ratio` to handle reordered terms and `partial_ratio` to catch cases where one phrase is contained inside a longer product description. The system keeps the best score from these comparisons. But even before scoring, the text is normalized, because, for example, German compound words are split where possible, so "Naturjoghurt" can be treated closer to "natur joghurt."

The third stage is the full catalogue fallback. If the best result from the customer template is below the confidence threshold = 85, the system searches the full catalogue. However, the full catalogue result only overrides the template match if it is strictly better. This prevents the system from replacing a reasonable customer-specific match with a weak full-catalogue result.

Review interface and feedback storage

The final output of the system is not sent directly to the ERP, but is displayed in a review interface first, so that if system is uncertain, a human should confirm the result before the order is finalized. The review interface shows each extracted item, the matched product, the item code, the quantity, the confidence score, and alternative matches. It also includes a threshold slider, so the reviewer can adjust how strict the system should be. Items below the threshold are highlighted in amber as well.

When an order is confirmed, it is saved in a SQLite database. At the current stage, this database mainly stores confirmed orders and corrections. However, this is important for future development. Over time, this can create a valuable dataset for improving the matcher.

Main challenges and how I solved them

Several challenges appeared during the development process:

1. The original extraction library and model reference in the starter code were not fully compatible with the current API setup. I had to update the integration and switch to a supported Gemini version.
2. The second challenge was input messiness. The orders contained mixed German and Italian words, abbreviations, spelling differences, and compound terms. A simple extraction and matching pipeline would not be enough. I addressed this with input-type-specific prompts, normalization, compound splitting, and synonym mapping.
3. Next challenge was the weak matching accuracy with the initial string comparison method. The original approach was too sensitive to word order and exact spelling. It did not work well on real catalogue descriptions. Replacing it with rapidfuzz and using token-based metrics improved the matching logic because it made the comparison more flexible.
4. Searching the full product catalogue for every extracted item was too broad and increased the risk of wrong matches. The customer-template-first approach solved this by narrowing the search space before fuzzy scoring.
5. Some extracted items could not be matched confidently. Instead of treating this as a failure, I added confidence scores and a human review step. This makes the system safer because uncertain results are clearly visible to the reviewer.

Results

The prototype successfully processes text, PDFs, and images, extracting order items, matching them to catalogue products, assigning confidence scores, and sending results for human review. The system works best when products exist in the customer template, with normalization, synonym mapping, and fuzzy scoring ensuring high-confidence matches for items like “Philadelphia” or “Burro mini porzioni.” Customer templates make matching faster and more relevant, while the full catalogue fallback handles exceptional orders.

Overall, the prototype is practical for reducing manual work and highlighting uncertain cases. With continued use, human corrections can be stored to improve future performance. Over time, this dataset could support machine-learning-based matching, allowing the system to evolve beyond a rule-based prototype into a more adaptive, intelligent solution.

Conclusion

This project resulted in a working prototype for AI-assisted order processing at Foppa. The system accepts informal customer orders as images, PDFs, or text messages, extracts structured information with Gemini, matches extracted phrases through a three-stage customer-aware matching pipeline, and returns an ERP-ready JSON draft for human review.

What I liked most about working on this project was that the problem was very realistic. The difficulty was not artificial: real customers write messy messages, use mixed languages, make abbreviations, and send orders in formats that are convenient for them. The most interesting part for me was the matching logic, because it showed how much can be improved by combining simple algorithms with domain knowledge.

Although, the prototype can still be improved, the current version already provides a clear foundation. It shows how Foppa’s informal orders can be transformed into structured ERP-ready drafts, while keeping human review in the loop and preparing the system for future learning.